

Requisitos do Projeto de revisão

Parte 1: Estrutura e Semântica (HTML5)

Os alunos devem criar uma estrutura totalmente semântica contendo:

header: Com a logo da aplicação, título do dashboard e um indicador de status da conexão (Online/Offline).

main: Dividido em duas seções principais:

section (Filtros e Controles): Botão para atualizar os dados manualmente e um select para filtrar sensores por tipo (Temperatura, Umidade, Pressão).

section (Grid de Sensores): O container principal onde os cards dos sensores serão renderizados dinamicamente via JS.

footer: Informações de copyright e a hora do último timestamp de atualização.

Parte 2: Estilização e Responsividade (CSS3)

A interface deve ser moderna, limpa e responsiva:

Layout Principal: Utilize **CSS Grid** para a estrutura macro (Header, Main, Footer) e para a distribuição dos cards dos sensores na tela (mínimo de 3 colunas em telas desktop, passando para 1 coluna em dispositivos móveis usando media queries).

Cards dos Sensores: Utilize **Flexbox** interno para alinhar os elementos de cada card (ícone, valor atual, tipo de sensor e botão de histórico).

Estados Visuais: Criar classes CSS para destacar sensores com leituras críticas (ex: temperatura maior que 35°C deve pintar a borda ou o fundo do card com um tom de alerta/vermelho suave).

Design System: Definir variáveis CSS (:root) para a paleta de cores (modo

Parte 3: Lógica e Manipulação de Estado em Memória — *Tempo*

estimado: 2h

Em vez de buscar os dados de um servidor, você deverá gerenciar um **array de objetos estático** diretamente no código e simular as alterações de dados usando funções nativas do JavaScript.

O arquivo script.js deve conter as seguintes implementações:

Massa de Dados (Dataset Inicial)

No topo do arquivo, os alunos devem declarar uma constante com uma lista de sensores (mínimo de 6 para que o layout Grid e os filtros façam sentido):

```
// JavaScript
```

```
const sensoresIniciais = [
```

```
{ id: 1, nome: "Sensor Galpão A", tipo: "Temperatura", valor: 24.5, unidade: "°C", status: "normal" },
```

```
{ id: 2, nome: "Sensor Estufa 02", "tipo": "Umidade", valor: 88.0, unidade: "%", status: "critico" },
```

```
{ id: 3, nome: "Sensor Compressor", tipo: "Pressão", valor: 6.2, unidade: "bar", status: "normal" },
```

```
{ id: 4, nome: "Sensor Câmara Fria", tipo: "Temperatura", valor: -  
2.1, unidade: "°C", status: "normal" },  
  
{ id: 5, nome: "Sensor Almojarifado", tipo: "Umididade", valor:  
45.5, unidade: "%", status: "normal" },  
  
{ id: 6, nome: "Sensor Caldeira", tipo: "Temperatura", valor: 98.4,  
unidade: "°C", status: "critico" }  
  
];
```

Renderização Dinâmica do DOM

Criar uma função (ex: renderizarDashboard(listaSensores)) que recebe um array de sensores.

A função deve limpar o container principal (container.innerHTML = "") e percorrer o array gerando a estrutura HTML de cada card.

Regra de Negócio Visual: Dentro do laço de repetição, se o sensor estiver com o status: "critico", o script deve adicionar uma classe CSS específica (ex: .card-alerta) ao elemento do card.

Filtros em Memória

Adicionar um evento de escuta (change) no elemento select da página.

Ao mudar o filtro, o JavaScript deve usar o método .filter() no array sensoresIniciais para gerar uma nova lista contendo apenas os sensores do tipo selecionado (Temperatura, Umididade ou Pressão). Se a opção selecionada for "Todos", exibe o array completo.

Chamar a função de renderização passando essa nova lista filtrada.

Simulação de Atualização (Tempo Real)

Criar uma função para atualizar o rodapé com o horário atual formatado (HH:MM:SS) sempre que a tela mudar.

O Desafio do Código: Para substituir o botão de "Atualizar dados", você deve criar uma função que percorre o array de sensores e altera levemente os valores numéricos (valor) usando `Math.random()`. Isso simulará os sensores mudando de valor na vida real.

Disparar essa função de atualização tanto no clique do botão "Atualizar" quanto automaticamente a cada 30 segundos usando `setInterval`.

Entregáveis do Aluno

Estrutura de arquivos padrão (index.html, style.css, script.js, sensores.json).

O código deverá ser publicado no GitHub.